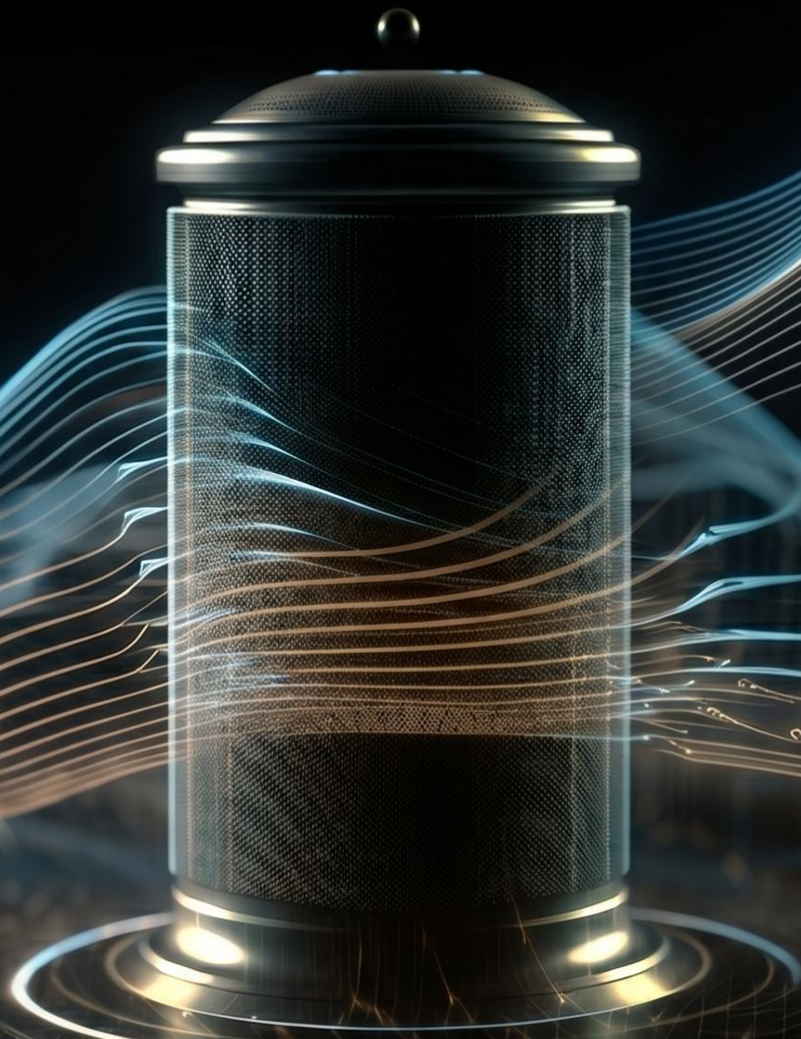


BATTERY MANAGEMENT SYSTEM

using Extended Kalman Filter (EKF)



Project Report

ABSTRACT

This project presents a Battery Management System (BMS) model developed in **MATLAB** for estimating the State of Charge (SOC) of a battery using an Extended Kalman Filter (EKF). Accurate SOC estimation is essential for improving battery performance, safety, and reliability in energy storage systems and electric vehicles.

The model simulates battery behaviour under dynamic current loading conditions while considering temperature variation, internal resistance, and measurement noise. A nonlinear Open Circuit Voltage (OCV)–SOC relationship is used to predict battery voltage. The EKF algorithm estimates SOC by comparing predicted voltage with measured voltage and correcting estimation errors.

The project also includes fault detection based on voltage deviation and evaluates estimation accuracy using Root Mean Square Error (RMSE). Simulation results show that the EKF provides reliable SOC estimation with low error and effective tracking of battery behaviour under varying operating conditions. This work demonstrates a practical MATLAB-based approach for battery monitoring and estimation applications.

INTRODUCTION

Battery Management Systems (BMS) play an important role in monitoring and controlling rechargeable batteries used in applications such as electric vehicles, renewable energy storage systems, portable electronics, and backup power systems. A BMS ensures safe and efficient battery operation by monitoring key parameters such as voltage, current, temperature, and State of Charge (SOC).

State of Charge (SOC) represents the remaining available energy in a battery and is usually expressed as a percentage. Accurate SOC estimation is essential because it helps prevent overcharging, deep discharge, reduced battery life, and unsafe operating conditions. However, SOC cannot be measured directly and must be estimated using mathematical models and estimation techniques.

Traditional SOC estimation methods such as Coulomb counting accumulate error over time due to sensor noise and model uncertainties. To improve estimation accuracy, advanced algorithms such as the Extended Kalman Filter (EKF) are widely used. EKF is suitable for nonlinear systems and provides improved estimation by continuously correcting prediction errors using measured voltage data.

In this project, a MATLAB-based Battery Management System model is developed to estimate battery SOC using EKF. The model includes dynamic current input, voltage prediction using an Open Circuit Voltage (OCV) relationship, temperature variation, fault detection, and estimation error analysis. The project demonstrates how EKF can improve battery monitoring performance under varying operating conditions.

OBJECTIVES

The main objectives of this project are:

- To develop a Battery Management System (BMS) model using **MATLAB**.
- To estimate the State of Charge (SOC) of a battery using the Extended Kalman Filter (EKF).
- To simulate battery behaviour under dynamic current loading conditions.
- To model battery voltage using an Open Circuit Voltage (OCV)–SOC relationship.
- To include temperature variation effects in battery voltage estimation.
- To compare measured voltage with predicted voltage for estimation correction.
- To implement fault detection based on voltage deviation.
- To evaluate estimation performance using Root Mean Square Error (RMSE).
- To analyse battery performance through graphical visualization of SOC, voltage, current, and estimation error.

MATHEMATICAL MODEL

The mathematical model describes the relationship between battery parameters such as State of Charge (SOC), voltage, current, and temperature. These equations are used in the Battery Management System (BMS) to estimate battery behaviour and implement the Extended Kalman Filter (EKF).

1. State of Charge (SOC) Model

SOC represents the remaining battery capacity and is calculated using the Coulomb Counting method.

The SOC update equation is:

$$SOC(k) = SOC(k - 1) - \frac{I(k) \times dt}{Capacity \times 3600} \times 100$$

Where:

- $SOC(k)$ = Current SOC (%)
- $SOC(k - 1)$ = Previous SOC (%)
- $I(k)$ = Battery current (A)
- dt = Sampling time (s)
- Capacity = Battery capacity (Ah)
- 3600 = Conversion from hours to seconds

This equation updates SOC at every simulation step based on current

2. Battery Voltage Model (OCV-SOC Relationship)

Battery voltage is modelled using an Open Circuit Voltage (OCV) equation dependent on SOC and temperature.

$$V_{OCV} = 3 + 1.2 \left(\frac{SOC}{100} \right) - 0.3 \left(\frac{SOC}{100} \right)^2 - 0.002(T - 25)$$

Where:

- V_{OCV} = Open Circuit Voltage (V)
- SOC = State of Charge (%)
- T = Temperature (°C)

This nonlinear equation approximates battery voltage variation with SOC.

3. Terminal Voltage Model

The actual terminal voltage includes internal resistance drop:

$$V = V_{OCV} - I \times R_0$$

Where:

- V = Battery terminal voltage (V)
- R_0 = Internal resistance (Ω)
- I = Battery current (A)

This accounts for voltage loss due to internal resistance.

4. Temperature Model

Battery temperature is varied using a sinusoidal function:

$$T = 25 + 5\sin(0.001t)$$

Where:

- T = Temperature ($^{\circ}\text{C}$)
- t = Simulation time (s)

This simulates changing operating temperature.

5. Extended Kalman Filter (EKF) Model

The EKF estimates SOC using prediction and correction stages.

Predicted Equation-

$$SOC_{pred} = SOC_{est}(k-1) - \frac{I(k) \times dt}{Capacity \times 3600} \times 100$$

Covariance Prediction-

$$P = FPF^T + Q$$

Where:

- P = Covariance Matrix
- F = State Jacobian
- Q = Process Noise Covariance

6. Measurement Model

Measured Voltage:

$$V_{measured} = V_{OCV} - I \times R_0 + Noise$$

Predicted Voltage:

$$V_{predicted} = V_{OCV}(SOC_{pred}) - I \times R_0$$

7. Jacobian Matrix

The Measurement Jacobian is:

$$H = \frac{dV}{dSOC}$$

For the Voltage Equation:

$$H = \frac{1.2}{100} - \frac{2 \times 0.3}{100} \left(\frac{SOC}{100} \right)$$

8. Kalman Gain

Kalman Gain determines correction strength:

$$K = \frac{PH^T}{HPH^T + R}$$

Where:

- K = Kalman Gain po
- R = Measurement noise covariance

9. EKF Update Equation

Final SOC estimation:

$$SOC_{est} = SOC_{pred} + K (V_{measured} - V_{predicted})$$

This corrects predicted SOC using measurement error.

10. Fault Detection Model

Fault is detected using voltage deviation:

$$|V_{measured} - V_{predicted}| > 0.2$$

If the condition is true:

- Fault = 1
- Otherwise, Fault = 0

This identifies abnormal battery behaviour.

MATLAB IMPLEMENTATION

The Battery Management System (BMS) model is implemented in MATLAB using an Extended Kalman Filter (EKF) for State of Charge (SOC) estimation. The MATLAB script simulates battery operation under dynamic loading conditions and estimates SOC by comparing measured and predicted battery voltage.

The implementation is divided into several stages:

1. Initialization of Parameters

Battery parameters and simulation settings are initialized at the beginning of the program.

These include:

- Battery capacity
- Internal resistance
- Sampling time
- Simulation duration
- EKF covariance parameters

Example:

```
Capacity = 1;      % Battery Capacity (Ah)
dt = 1;           % Time Step (sec)
R0 = 0.015;      % Internal Resistance (Ohm)
```

2. Dynamic Current Generation

A sinusoidal current profile with random noise is generated to simulate varying battery load conditions.

$$Current = 3 \times \sin(\omega \times time) + 0.3 \times randn(size(time));$$

This creates a realistic charging and discharging current platform.

3. Temperature Modelling

Temperature variation is introduced to simulate changing operating conditions.

$$T = 25 + 5 \times \sin(\omega \times time);$$

Temperature affects battery voltage estimation.

4. SOC Initialization

Arrays are created to store:

- True SOC
- Estimated SOC
- Fault detection values

Initial SOC is assumed to be 100%.

$$socTrue(1) = 100;$$

$$socEst(1) = 100;$$

5. Voltage Model Definition

A nonlinear Open Circuit Voltage (OCV) equation is implemented using an anonymous function.

$$ocv = (soc, T) \rightarrow 3 + 1.2 \times \frac{SOC}{100} - 0.3 \times \left(\frac{SOC}{100}\right)^2 - 0.002 \times (T - 25)$$

This models battery voltage as a function of SOC and temperature.

6. EKF Prediction Step

The EKF predicts the next SOC value using Coulomb counting.

$$socPred = socEst(k-1) - \frac{current(k) \times dt}{Capacity \times 3600} \times 100$$

Covariance prediction:

$$P = FPF' + Q;$$

7. Voltage Measurement Simulation

Measured battery voltage is generated using the voltage model with added noise.

$$Voltage_{meas} = ocv(socTrue(k), T(k)) - current(k) \times R_0 + 0.05 \times randn;$$

8. EKF Update Step

The EKF calculates Kalman Gain and updates the SOC estimate.

Kalman Gain:

$$K = \frac{PH'}{HPH' + R};$$

SOC Update:

$$socEst(k) = socPred + K(Voltage_{meas} - Voltage_{pred});$$

9. Fault Detection

Fault detection is performed by comparing measured and predicted voltage.

$$fault(k) = abs(Voltage_{meas} - Voltage_{pred}) > 0.2;$$

If, Voltage Difference exceeds threshold, a fault is flagged.

10. Capacity Fading

Battery aging is simulated by reducing battery capacity gradually.

$$Capacity = Capacity \times 0.99999;$$

11. Error Calculation

Estimation Error is calculated:

$$Error = |socTrue - socEst|;$$

RMSE is computed to evaluate estimation performance.

$$RMSE = \sqrt{\text{mean}(\text{error}^2)};$$

12. Data Visualisation

The results are displayed using tiled plots.

Plots include:

- SOC estimation
- Current profile
- Voltage comparison
- Estimation error
- Fault detection

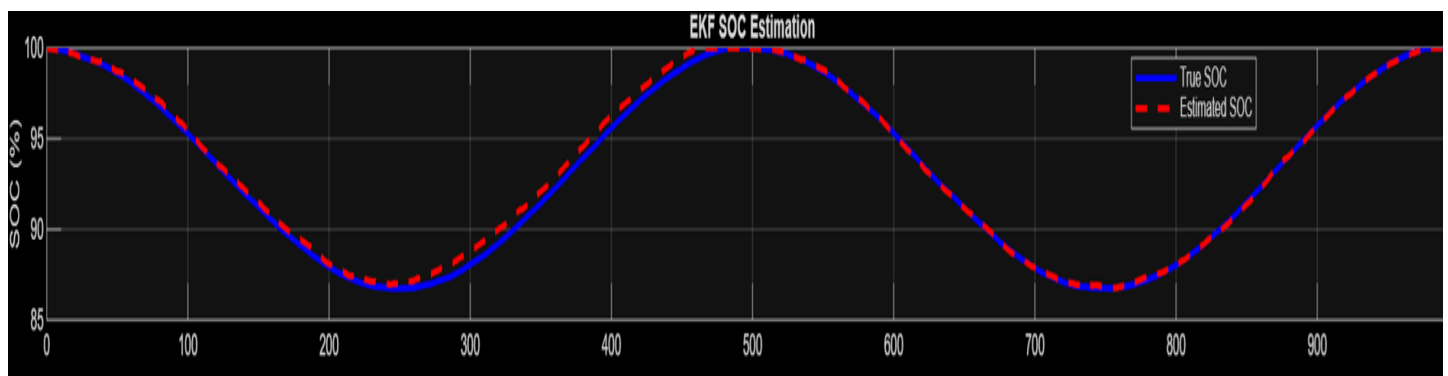
This provides a complete analysis of battery performance and EKF estimation accuracy.

RESULTS & GRAPHS

The simulation results obtained from the **MATLAB** implementation demonstrate the performance of the Extended Kalman Filter (EKF) for battery State of Charge (SOC) estimation. Multiple plots are generated to analyse battery behaviour, estimation accuracy, and fault detection.

1. SOC Estimator

This graph compares the **True SOC** and **Estimated SOC** over simulation time.



Observation:

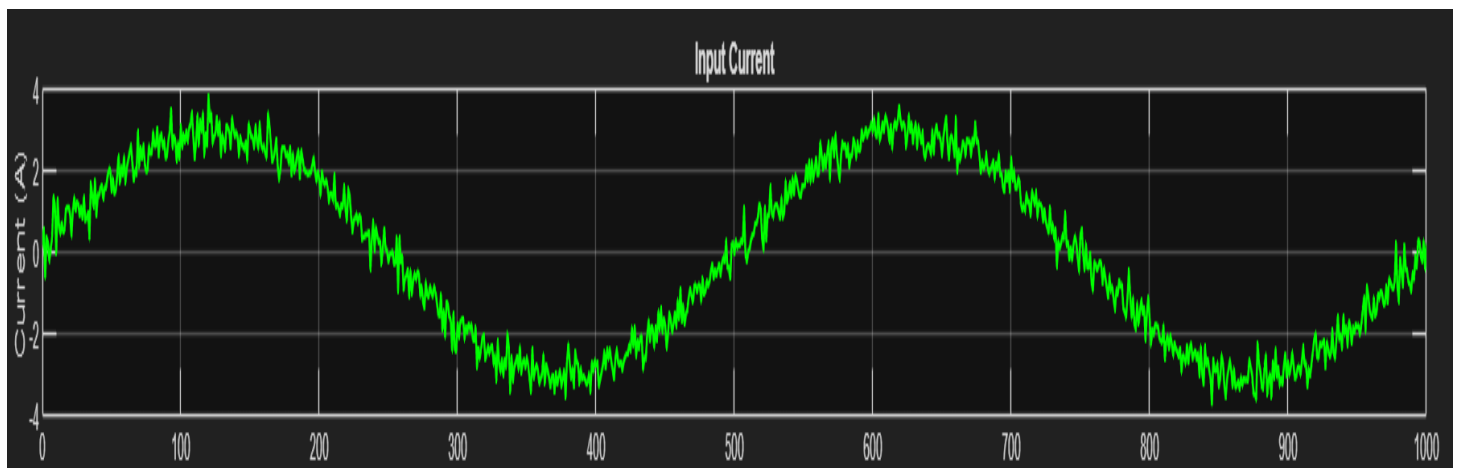
- The estimated SOC closely follows the true SOC.
- EKF continuously corrects prediction error.
- Small deviation exists due to measurement noise.

Purpose:

- Validates the accuracy of EKF-based SOC estimation.

2. Current Profile Plot

This graph shows the dynamic input current applied to the battery.



Observation:

- Current varies sinusoidally with added random noise.
- Positive current represents discharge condition.
- Negative current represents charging condition.

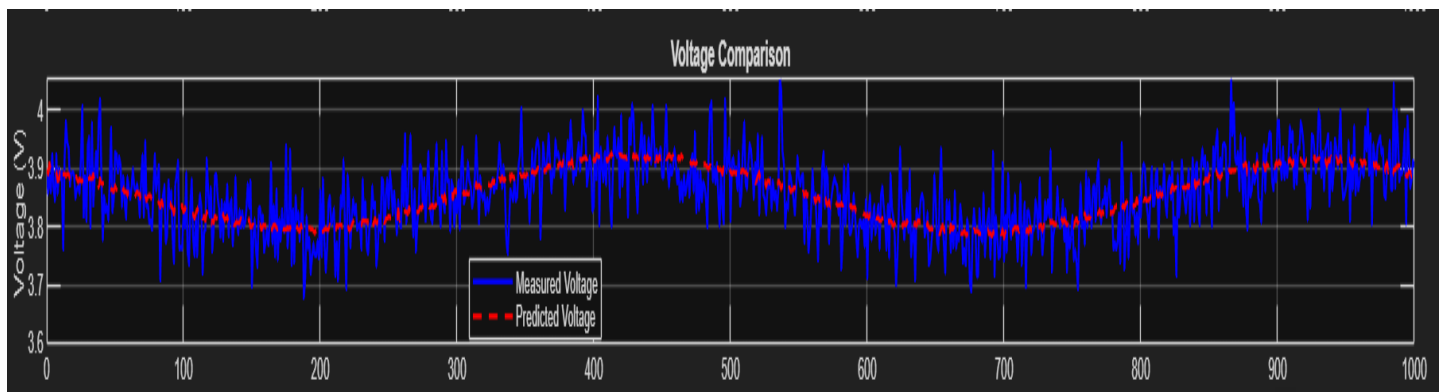
Purpose:

- Simulates realistic battery operating conditions.

3. Voltage Comparison Plot

This graph compares:

- Measured Voltage
- Predicted Voltage

**Observation:**

- Predicted voltage follows measured voltage closely.
- Small fluctuations occur due to measurement noise.
- Voltage variation changes with SOC and temperature.

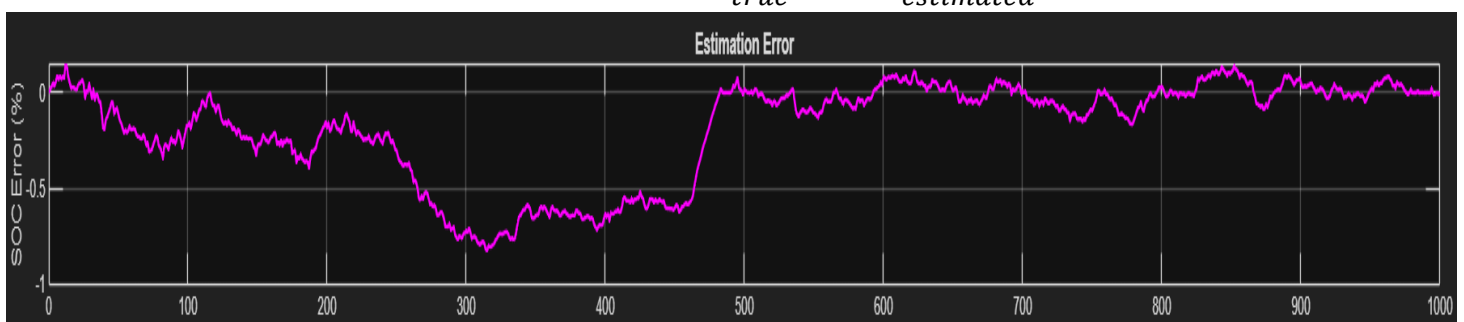
Purpose:

- Demonstrates EKF voltage tracking performance.

4. Estimation Error Plot

This graph displays the difference between true SOC and estimated SOC.

$$Error = SOC_{true} - SOC_{estimated}$$



Observation:

- Error remains small throughout simulation.
- EKF minimizes estimation deviation.
- Stable error indicates good filter tuning.

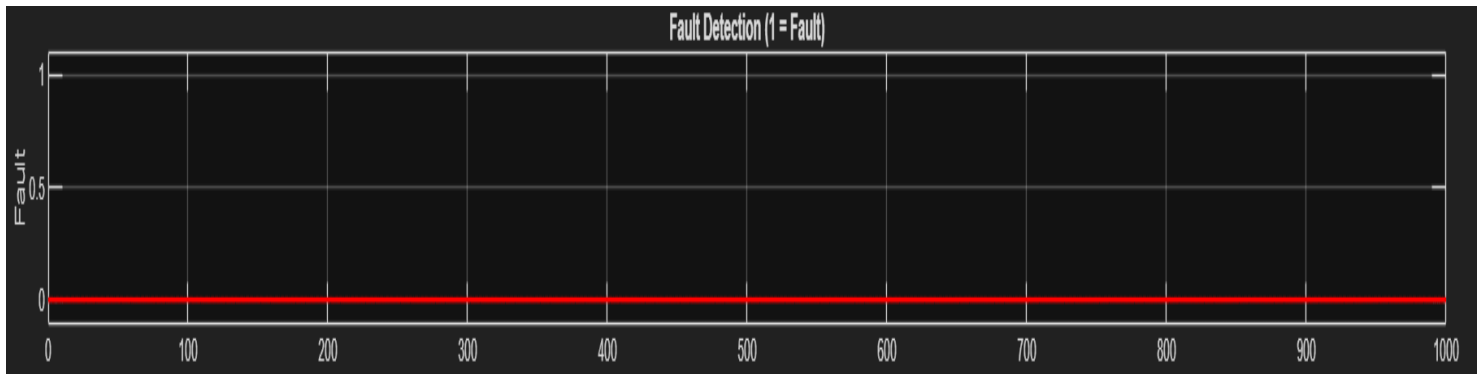
Purpose:

- Measures estimation quality.

5. Fault Detection Plot

This graph indicates detected faults using binary values:

- 0 → No Fault
- 1 → Fault Detected

**Observation:**

- Faults appear when measured voltage deviates significantly from predicted voltage.
- Threshold-based detection identifies abnormal behaviour.

Purpose:

- Helps monitor battery safety.

6. RMSE Result

Root Mean Square Error (RMSE) is used to evaluate estimation accuracy.

$$RMSE = \left[\frac{1}{N} \sum (error^2) \right]^{0.5}$$

Observation:

- Lower RMSE indicates higher estimation accuracy.
- RMSE confirms EKF performance.

Output:

$$RMS\ Error = 0.17934$$

Overall Result Summary

The MATLAB simulation successfully demonstrates:

- Accurate SOC estimation using EKF
- Reliable voltage prediction
- Stable estimation error
- Effective fault detection
- Realistic battery behaviour simulation

The generated plots validate the effectiveness of the Battery Management System model under varying current and temperature conditions.

CONCLUSION

This project successfully demonstrates the implementation of a Battery Management System (BMS) using an Extended Kalman Filter (EKF) in **MATLAB** for accurate State of Charge (SOC) estimation. The developed model simulates battery behaviour under dynamic current loading conditions while considering temperature variation, internal resistance, and measurement noise.

The EKF algorithm effectively predicts and corrects SOC values by comparing measured voltage with predicted voltage. Simulation results show that the estimated SOC closely follows the true SOC, resulting in low estimation error and reliable performance. The inclusion of fault detection further improves system monitoring by identifying abnormal voltage deviations.

The generated plots for SOC estimation, voltage comparison, estimation error, and fault detection validate the effectiveness of the proposed model. Overall, this project provides a practical understanding of battery monitoring techniques and demonstrates how EKF can improve SOC estimation accuracy in Battery Management Systems.

REFERENCES

1. **MATLAB Documentation** – Used for MATLAB functions, plotting tools, and programming syntax.
2. **Simulink Documentation** – Reference for battery modelling concepts and simulation workflows.
3. Research papers on Battery Management Systems (BMS) and State of Charge (SOC) estimation.
4. Literature related to Extended Kalman Filter (EKF) theory for nonlinear estimation.
5. Battery modelling concepts from Electrical Engineering textbooks and academic resources.
6. Open Circuit Voltage (OCV) and SOC relationship studies used in battery estimation models.
7. Online technical tutorials, AI platforms and educational resources related to EKF implementation in battery systems.
8. Standard battery modelling and estimation concepts used in Electric Vehicle battery systems.